

Practical No. 7

TITLE: Group “B”

AIM: To implement nested queries and set operations (UNION, INTERSECT, MINUS) for complex data retrievals in a relational database.

THEORY

In a Relational Database Management System (RDBMS), complex data retrieval often requires combining results from multiple tables or using the output of one query as the input for another. This is achieved through **Nested Queries** and **Set Operations**.

TOOLS/ENVIRONMENT:

Sr. No	Name of software used	Version
1	Oracle Database / SQL Developer	Oracle Database 23ai

1. Nested Queries (Subqueries)

A subquery is a query nested within the **WHERE**, **HAVING**, or **FROM** clause of another SQL query. It allows for dynamic filtering where the criteria are not known upfront but are calculated at runtime.

- **Subquery Structure:**

- **Inner Query:** The subquery that executes first and returns a result to the outer query.
- **Outer Query:** The main query that uses the result of the subquery to produce the final output.

- **Types of Subqueries :-**

- **Single-row Subquery:** Returns only one row from the inner select statement. It uses operators like =, >, <.

Example: Find students with marks greater than the student with Roll No 101.

SQL

```
SELECT Name FROM STUDENTS
WHERE Marks > (SELECT Marks FROM ENROLLMENT WHERE Roll_No =
101);
```

- **Multiple-row Subquery:** Returns more than one row of data. It uses operators like IN, ANY, ALL, or EXISTS.

Example: Find students enrolled in the same departments as 'Rahul'.

SQL

```
SELECT Name FROM STUDENTS
WHERE Dept_ID IN (SELECT Dept_ID FROM STUDENTS WHERE Name =
'Rahul');
```

- **Correlated Subquery:** A subquery that refers to a column from the outer query. It is evaluated once for every row processed by the outer query.

Example: Find students whose marks are greater than the average marks of their own department.

```
SELECT S.Name FROM STUDENTS S
WHERE S.Marks > (SELECT AVG(E.Marks) FROM ENROLLMENT E
WHERE E.Roll_No = S.Roll_No);
```

Non-Correlated Subquery: A subquery that can be executed independently of the outer query.

Example: Find names of students in 'Dept 10'.

SQL

```
SELECT Name FROM STUDENTS
WHERE Dept_ID = (SELECT Dept_ID FROM COURSES WHERE Dept_ID =
10);
```

2. Set Operations

Set operations allow us to combine the results of two or more independent **SELECT** statements into a single result set. For these operations to be valid, both queries must have the same number of columns and compatible data types in the same order.

- **UNION:** Combines the results of two queries and removes duplicate rows.
- **UNION ALL:** Combines results but retains all duplicates.
- **INTERSECT:** Returns only the common records found in both result sets.
- **MINUS (or EXCEPT):** Returns records from the first query that are not present in the second query.

IMPLEMENTATION & EXAMPLES

To demonstrate these concepts, consider the following schema:

- **STUDENTS** (Roll_No, Name, Dept_ID)
- **COURSES** (Course_ID, Course_Name, Dept_ID)
- **ENROLLMENT** (Roll_No, Course_ID, Marks)

PART A – NESTED QUERIES

Objective: Find the names of students who have scored more than the average marks in the 'DBMS' course.

SQL Code:

SQL

SELECT Name

FROM STUDENTS

WHERE Roll_No IN (

 SELECT Roll_No

 FROM ENROLLMENT

 WHERE Marks > (SELECT AVG(Marks) FROM ENROLLMENT)

```
    AND Course_ID = (SELECT Course_ID FROM COURSES WHERE Course_Name =  
'DBMS')  
);
```

Explanation :

1. The innermost subquery finds the **AVG(Marks)**.
2. The second subquery finds the **Course_ID** for 'DBMS'.
3. The outer query then filters the **STUDENTS** based on these dynamically retrieved IDs and values.

PART B – SET OPERATIONS

1. UNION Operation

Objective: List all unique Department IDs currently associated with either Students or Courses.

SQL

```
SELECT Dept_ID FROM STUDENTS
```

```
UNION
```

```
SELECT Dept_ID FROM COURSES;
```

2. INTERSECT Operation

Objective: Find Department IDs that have both registered Students AND registered Courses.

SQL

```
SELECT Dept_ID FROM STUDENTS
```

```
INTERSECT
```

```
SELECT Dept_ID FROM COURSES;
```

3. MINUS Operation

Objective: Find Departments that have Courses listed but currently have no Students enrolled.

SQL

```
SELECT Dept_ID FROM COURSES
```

```
MINUS
```

```
SELECT Dept_ID FROM STUDENTS;
```

IMPORTANT CLAUSES IN NESTED QUERIES

1. **IN Operator** : Used when the subquery returns multiple values.
 - *Example:* `WHERE Dept_ID IN (10, 20, 30)`
 2. **EXISTS Operator**: Returns TRUE if the subquery returns at least one row. It is highly efficient for checking existence without retrieving data.
 - *Example:* `SELECT Name FROM STUDENTS S WHERE EXISTS (SELECT 1 FROM ENROLLMENT E WHERE E.Roll_No = S.Roll_No);`
 3. **ANY / ALL Operators**: * > **ANY**: Greater than at least one value in the list.
 - > **ALL**: Greater than every value in the list (i.e., greater than the maximum).
-

CONCLUSION

In this practical, we successfully implemented complex data retrieval techniques. We observed how Nested Queries allow for multi-level filtering and how Set Operations (UNION, INTERSECT, MINUS) facilitate the mathematical combination of distinct data sets. These tools are vital for generating advanced business intelligence reports from relational databases.

VIVA QUESTIONS

1. **What is the difference between a Correlated and Non-correlated subquery?**
2. **Can we use the ORDER BY clause inside a subquery?**
3. **What is the difference between UNION and UNION ALL?**
4. **Which Set Operator is used to find common records between two tables?**